

```

        have.insert(end(have), begin(a[i]), end(a[i]));
        vector<string>().swap(a[i]);
        aho[i] = AC();
    }
}

ll cnt(const string & s){
    ll answer = 0;
    for(int i = 0; i < lg; i++){
        int state = 0;
        for(auto & v : s){
            state = aho[i].advance(state, v);
            answer += aho[i].cnt[state];
        }
    }
    return answer;
}
};

```

Generalized suffix automaton.cpp

Use: GSAM g(total_len, m); g.init(total_len,m), g.build(strings); lowercase strings and m <= 63 for mask-based methods.

Apps: get_lcs(K) LCS length for first K strings; distinct_substrings() count; match(P) total occurrences; get_kth(K) kth distinct substring.

Apps: shortest_distinguishing() gives min unique length.

Apps: get_all_occurrences(P) gives {string_id,start}; string_specific_occ(P,i) counts in strings[i].

Care: Call build_tree() through build(); mask-based LCS/distinguishing routines need at most 63 strings.

Complexity: Build is linear in total length; occurrence-heavy routines may traverse suffix-link subtrees.

Pitfall: Masks use ll, so K beyond 63 strings needs a different bitset representation.

```

struct GSAM{
    static const int A = 26;
    struct node{
        int len, link;
        bool is_cloned;
        int nxt[A];
        node(){
            memset(nxt, -1, sizeof nxt);
            len = 0;
            link = -1;
            is_cloned = false;
        }
    };
    int sz;
    vector<node> t;
    vector<int> cnt;
    vector<vector<int>> inv, g;
    vector<ll> freq;
    GSAM(){}
    GSAM(int n, int m){
        init(n, m);
    }
    void init(int n, int m){
        t.assign(2 * n + 5, node());
        cnt.assign(2 * n + 5, 0);
        freq.assign(2 * n + 5, -1);
        g.assign(2 * n + 5, {});
    }
};

```

```

        inv.assign(m, {});
        sz = 1;
    }
    int get(char c){
        return c - 'a';
    }
    int extend(char c, int last){
        int c_idx = get(c);
        if (t[last].nxt[c_idx] != -1){
            int q = t[last].nxt[c_idx];
            if (t[last].len + 1 == t[q].len) return q;
            int clone = sz++;
            t[clone] = t[q];
            t[clone].len = t[last].len + 1;
            t[clone].is_cloned = true;
            int p = last;
            while (p != -1 && t[p].nxt[c_idx] == q){
                t[p].nxt[c_idx] = clone;
                p = t[p].link;
            }
            t[q].link = clone;
            return clone;
        }
        int cur = sz++;
        t[cur].len = t[last].len + 1;
        int p = last;
        while (p != -1 && t[p].nxt[c_idx] == -1){
            t[p].nxt[c_idx] = cur;
            p = t[p].link;
        }
        if (p == -1){
            t[cur].link = 0;
        }else{
            int q = t[p].nxt[c_idx];
            if (t[p].len + 1 == t[q].len){
                t[cur].link = q;
            }else{
                int clone = sz++;
                t[clone] = t[q];
                t[clone].len = t[p].len + 1;
                while (p != -1 && t[p].nxt[c_idx] == q){
                    t[p].nxt[c_idx] = clone;
                    p = t[p].link;
                }
                t[q].link = t[cur].link = clone;
                t[clone].is_cloned = true;
            }
        }
        return cur;
    }
    void build_tree(){
        for (int i = 1; i < sz; i++){
            g[t[i].link].push_back(i);
        }
    }
    void build(const vector<string> &strings){
        for (int i = 0; i < (int)strings.size(); i++){
            int lst = 0;
            inv[i].resize(strings[i].size());

```

```

        for (int j = 0; j < (int)strings[i].size(); j++){
            lst = extend(strings[i][j], lst);
            cnt[lst]++;
            inv[i][j] = lst;
        }
    }
    build_tree();
}
ll occ(int i){
    if (~freq[i]) return freq[i];
    ll ret = cnt[i];
    for (auto x : g[i]){
        ret += occ(x);
    }
    return freq[i] = ret;
}
void getMask(int u, vector<ll> &mask){
    for (int v : g[u]){
        getMask(v, mask);
        mask[u] |= mask[v];
    }
}
int get_lcs(int K){
    vector<ll> mask(sz, 0);
    for (int i = 0; i < (int)inv.size(); i++){
        for (int j = 0; j < (int)inv[i].size(); j++){
            mask[inv[i][j]] |= (1LL << i);
        }
    }
    getMask(0, mask);
    int max_len = 0;
    ll target = (1LL << K) - 1;
    for (int i = 1; i < sz; i++){
        if ((mask[i] & target) == target){
            max_len = max(max_len, t[i].len);
        }
    }
    return max_len;
}
ll distinct_substrings(){
    ll total = 0;
    for (int i = 1; i < sz; i++){
        total += t[i].len - t[t[i].link].len;
    }
    return total;
}
ll match(const string &P){
    int u = 0;
    for (char c : P){
        if (t[u].nxt[get(c)] == -1) return 0;
        u = t[u].nxt[get(c)];
    }
    return occ(u);
}
ll calc(int u, vector<ll> &dp){
    if (dp[u] != -1) return dp[u];
    dp[u] = 1;
    for (int c = 0; c < A; c++){
        if (t[u].nxt[c] != -1){

```

```

            dp[u] += calc(t[u].nxt[c], dp);
        }
    }
    return dp[u];
}
string get_kth(ll K){
    vector<ll> dp(sz, -1);
    calc(0, dp);
    string res = "";
    int u = 0;
    while (K > 0){
        for (int c = 0; c < A; c++){
            if (t[u].nxt[c] != -1){
                int v = t[u].nxt[c];
                if (dp[v] >= K){
                    res += (char)('a' + c);
                    u = v;
                    K--;
                    break;
                }else{
                    K -= dp[v];
                }
            }
        }
    }
    return res;
}
int shortest_distinguishing(){
    vector<ll> mask(sz, 0);
    for (int i = 0; i < (int)inv.size(); i++){
        for (int j = 0; j < (int)inv[i].size(); j++){
            mask[inv[i][j]] |= (1LL << i);
        }
    }
    getMask(0, mask);
    int min_len = 1e9;
    for (int i = 1; i < sz; i++){
        if (__builtin_popcountll(mask[i]) == 1){
            min_len = min(min_len, t[t[i].link].len + 1);
        }
    }
    return min_len == 1e9 ? -1 : min_len;
}
vector<pair<int, int>> get_all_occurrences(const string &P){
    int u = 0;
    for (char c : P){
        if (t[u].nxt[get(c)] == -1) return {};
        u = t[u].nxt[get(c)];
    }
    vector<vector<pair<int, int>>> state_endpoints(sz);
    for (int i = 0; i < (int)inv.size(); i++){
        for (int j = 0; j < (int)inv[i].size(); j++){
            state_endpoints[inv[i][j]].push_back({i, j});
        }
    }
    vector<pair<int, int>> res;
    collect(u, (int)P.length(), state_endpoints, res);
    return res;
}

```

```

void collect(int u, int len, vector<vector<pair<int, int>>> &ends, vector<pair<int,
↳ int>> &res){
    for (auto ep : ends[u]){
        res.push_back({ep.first, ep.second - len + 1});
    }
    for (int v : g[u]){
        collect(v, len, ends, res);
    }
}
ll dfsSp(int u, vector<int> &cnt2, vector<ll> &freq2){
    if (freq2[u] != -1) return freq2[u];
    ll ret = cnt2[u];
    for (int v : g[u]){
        ret += dfsSp(v, cnt2, freq2);
    }
    return freq2[u] = ret;
}
ll string_specific_occ(const string &P, int str_idx){
    int u = 0;
    for (char c : P){
        if (t[u].nxt[get(c)] == -1) return 0;
        u = t[u].nxt[get(c)];
    }
    vector<int> cnt2(sz, 0);
    for (int j = 0; j < (int)inv[str_idx].size(); j++){
        cnt2[inv[str_idx][j]]++;
    }
    vector<ll> freq2(sz, -1);
    return dfsSp(u, cnt2, freq2);
}
};

```

KMP.cpp

Use: KMP(s) builds pi; find_occurrences(pattern,text) returns all 0-indexed starts; automaton(pattern) builds transitions.

Care: find_occurrences concatenates pattern + '#' + text, so choose a separator not present if alphabet is not lowercase.

Complexity: KMP and find_occurrences are $O(n+m)$; automaton is $O(\text{pattern_length} * 26)$.

Pitfall: The automaton assumes lowercase a-z transitions.

```

vector<int> KMP(const string &s){
    int n = s.length();
    vector<int> pi(n);
    for (int i = 1; i < n; i++){
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]){
            j = pi[j - 1];
        }
        if (s[i] == s[j]) j++;
        pi[i] = j;
    }
    return pi;
}
vector<int> find_occurrences(const string &pat, const string &text){
    string s = pat + '#' + text;
    vector<int> pi = KMP(s), ret;
    int m = pat.size();
    for (int i = m + 1; i < (int)s.size(); i++){

```

```

        if (pi[i] == m) ret.push_back(i - 2 * m);
    }
    return ret;
}
vector<vector<int>> automaton(string s){ // aut[s.length() + 1][26]
    s += '#';
    int n = s.length();
    vector<int> pi = KMP(s);
    vector<vector<int>> aut(n, vector<int>(26));
    for (int i = 0; i < n; i++){
        for (int j = 0; j < 26; j++){
            if (i > 0 && 'a' + j != s[i]){
                aut[i][j] = aut[pi[i - 1]][j];
            }else{
                aut[i][j] = i + ('a' + j == s[i]);
            }
        }
    }
    return aut;
}

```

manacher.cpp

Use: manacher m(s); odd/even radii, ispal(l,r), longest_palindrome(), count_palindromes(), starts, and ends are built.

Out: starts[i] and ends[i] count palindromes starting/ending at i after build(), useful for palindrome DP counts.

Complexity: build is $O(n)$; palindrome checks and longest radii queries are $O(1)$.

Pitfall: ispal(l,r) expects inclusive 0-indexed bounds.

```

struct manacher{
    vector<int> p, odd, even;
    vector<ll> starts, ends;
    manacher(){
        build(S);
    }
    void run_manacher(const string &s){
        int n = s.length();
        p.assign(n, 1);
        int l = 1, r = 1;
        for (int i = 1; i < n; i++){
            p[i] = i < r ? min(r - i, p[l + r - i]) : 1;
            while (i - p[i] >= 0 && i + p[i] < n && s[i + p[i]] == s[i - p[i]]){
                p[i]++;
            }
            if (r < i + p[i]){
                r = i + p[i];
                l = i - p[i];
            }
        }
    }
    void build(const string &s){
        string t = "#";
        t.reserve(s.length() * 2 + 1);
        for (char c : s){
            t += c;
            t += '#';
        }
        run_manacher(t);
    }
}

```

6 DP Optimizations and Misc

divide and conquer dp.cpp

Use: Set n and k , edit $\text{add}(i)/\text{rem}(i)$ so $\text{cur}=\text{cost}(L,R)$, then $\text{compute}()$; $\text{answer}()$ returns $\text{dp}[n]$ after k layers.

Care: This maintains cost by moving $[L,R]$; add/rem may use current L/R exactly like Mo-style pointer movement.

Complexity: Each layer is $O(n \log n)$ divide-and-conquer states plus the amortized moving-window cost changes.

Recurrence: The template computes $\text{dp}[\text{mid}][\text{layer}] = \min \text{dp}[\text{split}-1][\text{layer}-1] + \text{cost}(\text{split}, \text{mid})$.

Pitfall: The optimization is only valid when optimal split points are monotone; otherwise use plain DP or another optimization.

```
const int N = 1e5 + 5;
const ll inf = (1LL << 62);

// dp[i][layer] = minimum cost to cover prefix [1..i] using this many layers.
// Transition shape: dp[mid][it] = min_{split <= mid} dp[split-1][it ^ 1] + cost(split,
// ↪ mid).
// Valid only when the optimal split point is monotone as mid increases.
int n, k, it = 1, L = 1, R = 0;
ll dp[N][2], cur = 0;

void add(int i){
    (void)i;
    // i has just entered the current interval [L, R].
    // Update cur so it equals cost(L, R).
}

void rem(int i){
    (void)i;
    // i is still inside [L, R] and is about to leave.
    // Update cur so it equals the interval after removing i.
}

void reset_window(){
    L = 1;
    R = 0;
    cur = 0;
}

void move(int l, int r){
    while(R < r) add(++R);
    while(L > l) add(--L);
    while(R > r) rem(R--);
    while(L < l) rem(L++);
}

void compute_layer(){
    vector<array<int, 4>> q;
    q.reserve(2 * n);
    q.push_back({1, n, 1, n}); // {answer range l,r, allowed opt range lx,rx}
    while(!q.empty()){
        auto [l, r, lx, rx] = q.back();
        q.pop_back();
        int mid = (l + r) >> 1, opt = lx;
        dp[mid][it] = inf;
        for(int split = lx; split <= min(rx, mid); split++){
            move(split, mid);
            ll val = dp[split-1][it ^ 1] + cur;
            if(val < dp[mid][it]){
                dp[mid][it] = val;
                opt = split;
            }
        }
    }
}
```

```
        if(l <= mid-1) q.push_back({l, mid-1, lx, opt});
        if(mid+1 <= r) q.push_back({mid+1, r, opt, rx});
    }
}

void compute(){
    for(int i = 1; i <= n; i++){
        dp[i][0] = dp[i][1] = inf;
    }
    dp[0][0] = dp[0][1] = 0;
    it = 1;
    for(int layer = 1; layer <= k; layer++){
        for(int i = 1; i <= n; i++) dp[i][it] = inf;
        reset_window();
        compute_layer();
        it ^= 1;
    }
}

ll answer(){
    return dp[n][it ^ 1];
}
```

Custom hashing.cpp

Use: Use $\text{unordered_map}<K,V,\text{custom_hash}>$ for integers or $\text{unordered_map}<\text{pair}<ll,ll>,V,\text{pair_hash}>$ for pairs to resist hacks.

Care: Use these hashes only in unordered containers; ordered maps/sets do not need them and cannot use them directly.

When: Use it for adversarial unordered_map/unordered_set tests with integer or pair keys.

Pitfall: Custom hash changes bucket distribution, not equality; pair equality still uses normal pair comparison.

```
struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }

    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM =
            chrono::steady_clock::now().time_since_epoch().count();
        return (size_t) splitmix64(x + FIXED_RANDOM);
    }
};

struct pair_hash {
    size_t operator()(const pair<ll,ll>& p) const noexcept {
        uint64_t a = (uint64_t) p.first;
        uint64_t b = (uint64_t) p.second;
        uint64_t z = a + 0x9e3779b97f4a7c15ULL + (b << 6) + (b >> 2);
        return (size_t) custom_hash::splitmix64(z);
    }
};
```

Josephus.cpp

Use: $\text{josephus_kth_removed}(n,k,m)$ returns the m -th removed 1-indexed label; survivor/order helpers cover common variants.

Care: $k=1$ is handled directly; $\text{josephus_order}(n,k)$ uses Fenwick and returns the whole deletion order in 1-indexed labels.

Complexity: $\text{kth_removed}/\text{survivor}$ use jump compression and are fast for large n ; josephus_order is $O(n \log n)$.

Out: survivor1 returns the 1-indexed survivor; survivor0_slow is the classic $O(n)$ 0-indexed recurrence.

```
// Returns the m-th removed person, 1-indexed labels, 1 <= m <= n.
// People are removed by counting k alive people starting from person 1.
```

```
11 josephus_kth_removed(11 n, 11 k, 11 m){
    assert(n >= 1 && k >= 1 && 1 <= m && m <= n);
    m = n - m;
    if (k == 1) return n - m;
    11 alive = m;
    while (alive < n){
        11 jump = (11)((__int128)alive - m + k - 2) / (k - 1));
        if ((__int128)alive + jump > n) jump = n - alive;
        else if (jump == 0) jump = 1;
        alive += jump;
        m = (11)((m + (__int128)jump * k) % alive);
    }
    return m + 1;
}

11 josephus_survivor1(11 n, 11 k){
    return josephus_kth_removed(n, k, n);
}

11 josephus_survivor0_slow(11 n, 11 k){
    assert(n >= 1 && k >= 1);
    11 ans = 0;
    for (11 i = 1; i <= n; i++){
        ans = (ans + k) % i;
    }
    return ans;
}
```

```
struct FenwickJosephus{
    int n;
    vector<int> bit;
    FenwickJosephus(){
        FenwickJosephus(int _n){ init(_n); }
        void init(int _n){
            n = _n;
            bit.assign(n + 1, 0);
            for (int i = 1; i <= n; i++) add(i, 1);
        }
        void add(int i, int v){
            for (; i <= n; i += i & -i) bit[i] += v;
        }
        int kth(int k){
            int pos = 0;
            for (int pw = 1 << __lg(n); pw; pw >>= 1){
                int nxt = pos + pw;
                if (nxt <= n && bit[nxt] < k){
                    pos = nxt;
                    k -= bit[nxt];
                }
            }
            return pos + 1;
        }
    };
    vector<int> josephus_order(int n, 11 k){
        assert(n >= 1 && k >= 1);
        FenwickJosephus fw(n);
        vector<int> order;
```

```
        order.reserve(n);
        11 cur = 0;
        for (int alive = n; alive >= 1; alive--){
            cur = (cur + k - 1) % alive;
            int pos = fw.kth((int)cur + 1);
            order.push_back(pos);
            fw.add(pos, -1);
        }
        return order;
}
```

Permutation cycles.cpp

Use: Helpers use 0-indexed permutations with $p[i]$ =image of i ; cycle order represents $v \rightarrow \text{next}(v)$.

decompose: perm_decompose(p) gets a permutation p and returns its disjoint cycles in orbit order; $O(n)$.

restore: perm_restore(n,cycles) gets cycle lists and returns p with each element mapped to the next element; $O(\text{total length})$.

power: perm_power_cycles(p,k) gets p and $k \geq 0$, then returns the cycle decomposition of p^k ; $O(n)$.

root: perm_root_cycles(p,k,toggle) gets p and $k > 0$, returns cycles of q with $q^k = p$ or empty if impossible.

swap list: perm_transpositions(cycles) returns swaps (c0,ci) for each cycle, count n-number_of_cycles; $O(\text{total length})$.

Root detail: For a length L cycle in p^k , preimages are formed by merging equal-length cycles according to $\text{gcd}(L,k)$.

Toggle: toggle tries one legal 2^* need merge to change transposition parity when the problem asks for a parity target.

Pitfall: The helpers do not validate that p is a permutation; duplicates or out-of-range images break the orbit assumptions.

```
vector<vector<int>> perm_decompose(const vector<int> &p){
    int n = p.size();
    vector<vector<int>> cycles;
    vector<bool> vis(n);
    for (int i = 0; i < n; i++){
        if (vis[i]) continue;
        vector<int> cycle;
        int u = i;
        while (!vis[u]){
            cycle.push_back(u);
            vis[u] = true;
            u = p[u];
        }
        cycles.push_back(cycle);
    }
    return cycles;
}

vector<int> perm_restore(int n, const vector<vector<int>> &cycles){
    vector<int> p(n);
    for (const auto &cycle : cycles){
        int m = cycle.size();
        for (int i = 0; i < m; i++){
            p[cycle[i]] = cycle[(i + 1) % m];
        }
    }
    return p;
}

vector<vector<int>> perm_power_cycles(const vector<int> &p, 11 k){
    assert(k >= 0);
    auto cycles = perm_decompose(p);
```

```

vector<vector<int>>> ans;
for (const auto &cycle : cycles){
    int len = cycle.size();
    int step = k % len;
    int g = gcd(step, len);
    for (int i = 0; i < g; i++){
        vector<int> cur;
        for (int j = i, cnt = 0; cnt < len / g; cnt++, j = (j + step) % len){
            cur.push_back(cycle[j]);
        }
        ans.push_back(cur);
    }
}
return ans;
}

vector<vector<int>>> perm_root_cycles(const vector<int> &p, ll k, bool toggle = false){
    assert(k > 0);
    int n = p.size();
    vector<vector<vector<int>>>> by_len(n + 1);
    for (auto &cycle : perm_decompose(p)){
        by_len[(int)cycle.size()].push_back(cycle);
    }
    vector<vector<int>>> ans;
    for (int len = 1; len <= n; len++){
        if (by_len[len].empty()) continue;
        ll tmp = k, need = 1;
        int g = gcd((ll)len, tmp);
        while (g != 1){
            need *= g;
            tmp /= g;
            g = gcd((ll)len, tmp);
        }
        if ((ll)by_len[len].size() % need != 0) return {};
        int id = 0, block = need;
        if (toggle && tmp % 2 == 0 && (ll)by_len[len].size() >= 2 * need){
            int merged = 2 * block;
            int sz = merged * len;
            vector<int> cycle(sz);
            for (int x = 0; x < merged; x++){
                for (int y = 0; y < len; y++){
                    cycle[(x + 1LL * y * k) % sz] = by_len[len][x][y];
                }
            }
            ans.push_back(cycle);
            id = merged;
            toggle = false;
        }
        for (int i = id; i < (int)by_len[len].size(); i += block){
            int sz = block * len;
            vector<int> cycle(sz);
            for (int x = 0; x < block; x++){
                for (int y = 0; y < len; y++){
                    cycle[(x + 1LL * y * k) % sz] = by_len[len][i + x][y];
                }
            }
            ans.push_back(cycle);
        }
    }
    return ans;
}

```

```

}
vector<pair<int, int>> perm_transpositions(const vector<vector<int>>> &cycles){
    vector<pair<int, int>> ans;
    for (const auto &cycle : cycles){
        for (int i = (int)cycle.size() - 1; i > 0; i--){
            ans.push_back({cycle[0], cycle[i]});
        }
    }
    return ans;
}

```